

Clase 2

Cómo funciona un LLM: tokens, embeddings y RAG

Guía de trabajo del aprendiz

Duración: 4 horas (8:00 am – 12:00 m)

Pablo Andrés Carvajal — Desarrollador Full Stack

pablocarvajal.dev

Nombre del aprendiz: _____

¿Qué sabré hacer al terminar esta guía?

- Explicar qué es un **token** y por qué el modelo no ve "palabras".
- Explicar qué es un **embedding** (vectorización) y qué es la similitud semántica.
- Describir a alto nivel qué hace un **Transformer** y el mecanismo de **atención**.
- Explicar cómo el LLM **genera texto** (predicción del siguiente token + temperatura).
- Entender **chunking** y **RAG**, y por qué sirven para darle datos propios al modelo.

LA GRAN IDEA DE HOY

Un modelo de IA **no entiende letras ni palabras: solo entiende números**. Toda esta guía explica cómo se traduce el lenguaje humano a números y cómo, con esos números, la máquina "entiende" y "escribe".

DEMOS INTERACTIVOS DE ESTA CLASE

Abre estas páginas en el navegador para **ver** los conceptos en vivo:

- **Cómo responde un LLM por probabilidad** (muestra cada palabra con su **% de certeza** y ajustas la **temperatura**): [Transformer Explainer](#). Versiones más sencillas: [LLM Sampling Visualizer](#) y [Write With Transformer](#).
- **Ver los tokens de una frase**: [OpenAI Tokenizer](#)
- **Embeddings en un mapa 3D**: [Embedding Projector](#) · **Juego de similitud**: [Semantris](#)
- **Transformer en 3D**: bbycroft.net/llm · **Explicación visual**: [FT: cómo funciona la IA generativa](#)

Todas juntas en el hub del curso: [recursos/demos-interactivos.html](#)

1. Tokenización: de texto a números

El primer paso siempre es convertir el texto en números. Ese proceso tiene dos pasos.

1.1 Partir el texto en tokens

Un **token** es un pedacito de texto: puede ser una palabra completa, parte de una palabra o un signo de puntuación. Piensa en el token como la **pieza mínima de LEGO** con la que el modelo arma y desarma el lenguaje.

Texto: "Aprender IA es genial"

Tokens: ["Apr", "ender", " IA", " es", " genial"] (ejemplo ilustrativo)

REGLA PRÁCTICA

En promedio **1 token $\approx$ 4 caracteres $\approx$ $\frac{3}{4}$ de una palabra** en inglés. En español suele ocupar un poco más. Palabras comunes ("perro") son 1 token; palabras raras o largas ("electroencefalografista") se parten en varios.

1.2 Cada token tiene un ID (un número)

El modelo tiene un **vocabulario**: una tabla enorme donde cada token único tiene asignado un número. Es como un diccionario en el que cada entrada tiene un código.

```
"Apr"      → 15496
"ender"    → 2938
" IA"      → 8712
" es"      → 318
" genial"  → 40021
```

Así, "Aprender IA es genial" se convierte internamente en la lista [15496, 2938, 8712, 318, 40021]. Eso es lo que "ve" el modelo.

1.3 Por qué esto importa en la práctica

Consecuencia	Qué significa para ti
Se cobra por token	Entrada + salida cuestan. Prompts más cortos y precisos ahorran dinero.
Límite de contexto	Hay un máximo de tokens por conversación (ej. 128.000). No cabe un libro infinito.
La IA a veces "olvida"	Si la charla es muy larga, el inicio se sale de la ventana de contexto.

PRÁCTICA SUGERIDA

Abre un tokenizador visual online (por ejemplo el "Tokenizer" de OpenAI), pega una frase en español y observa cómo se parte y cuántos tokens ocupa. Compara "perro" (1 token) con "electroencefalografista" (varios).

2. Vectorización / Embeddings: el mapa del significado

Convertir a IDs no basta: el número `8712` no dice **nada** sobre el significado. Necesitamos que palabras **parecidas** tengan números **parecidos**. Eso son los **embeddings**.

2.1 ¿Qué es un embedding?

Un **embedding** es una **lista de números (un vector)** que representa el **significado** de un texto. En vez de un solo número, cada token o frase se convierte en un vector de cientos o miles de dimensiones.

```
"perro" → [0.21, -0.98, 0.44, ..., 0.10]    (ej. 1024 números)
"gato"  → [0.19, -0.95, 0.40, ..., 0.12]    ← ¡muy parecido a "perro"!
"avión" → [-0.80, 0.33, 0.91, ..., -0.55]   ← muy diferente
```

2.2 El significado como distancia

Imagina un **mapa gigante** (como el mapa de una ciudad) donde cada palabra es un punto. Las cosas con significado parecido quedan **cerca**; las diferentes, **lejos**.

```
gato • • perro
conejo •      • lobo

                • avión
                • helicóptero
```

"gato" y "perro" están cerca (ambos mascotas); "avión" está lejos (otro tema). La **similitud semántica** es qué tan cerca están dos vectores, y se mide con la **similitud coseno**: un número entre -1 y 1, donde cerca de 1 significa "muy parecidos".

LO MÁGICO

Los embeddings capturan **relaciones**. El ejemplo clásico: `rey - hombre + mujer ≈ reina`. El modelo aprende estas relaciones **solo**, a partir de los datos (aprendizaje autosupervisado que vimos en la Clase 1).

2.3 Práctica con Cohere

Ya tienes el archivo `clase-2/codigo/embeddings_similitud.py`. Instala lo necesario y ejecútalo:

```
pip install cohere numpy python-dotenv
python embeddings_similitud.py
```

El corazón del programa vectoriza unas palabras y mide su similitud coseno:

```
textos = ["perro", "gato", "avión", "cachorro"]
vectores = co.embed(texts=textos, model="embed-multilingual-v3.0",
                    input_type="search_document").embeddings

def similitud(a, b):
    a, b = np.array(a), np.array(b)
    return float(np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b)))

print("perro vs gato:      ", round(similitud(vectores[0], vectores[1]), 3))
print("perro vs cachorro:", round(similitud(vectores[0], vectores[3]), 3))
print("perro vs avión:    ", round(similitud(vectores[0], vectores[2]), 3))
```

RESULTADO ESPERADO

"perro vs cachorro" y "perro vs gato" dan valores altos; "perro vs avión" da un valor bajo. ¡El computador acaba de **medir significado**!

3. Transformers y atención: entender el contexto

Los LLMs usan una arquitectura llamada **Transformer** (2017, del paper "*Attention is All You Need*"). Su superpoder es el mecanismo de **atención (attention)**.

3.1 El problema que resuelve la atención

El significado de una palabra **depende del contexto**:

- "Fui al **banco** a sacar plata." → institución financiera.
- "Me senté en el **banco** del parque." → asiento.

La misma palabra, significados distintos. ¿Cómo lo sabe el modelo? **Mirando las otras palabras de la frase.**

3.2 Qué hace la atención (intuición)

Para cada palabra, el modelo se pregunta: "*¿a cuáles otras palabras debo prestar atención para entender esta?*" y les asigna un **peso**. Es como leer una frase y subrayar las palabras que te aclaran el sentido de otra.

"Me senté en el banco del parque"

▲
└─ presta MUCHA atención a "senté" y "parque"
→ concluye: banco = asiento

IDEA A RETENER

El Transformer procesa **todas las palabras a la vez** (no una por una), por eso es rápido y capta relaciones a larga distancia. Apila muchas capas de atención para entender relaciones cada vez más abstractas. No necesitas la matemática interna: quédate con que **entiende cada palabra en función de todas las demás**.

4. Cómo genera lenguaje natural

4.1 Una máquina de predecir el siguiente token

Un LLM hace **una sola cosa**, increíblemente bien: dado un texto, **predice cuál es el token más probable que sigue**. Luego lo agrega al texto y **repite**.

```
Entrada: "El cielo es de color"
El modelo calcula probabilidades del SIGUIENTE token:
  "azul" → 78%
  "gris" → 12%
  "rojo" → 4%
  "verde" → 1%
Elige "azul" → ahora la entrada es "El cielo es de color azul"
→ y vuelve a predecir el siguiente token, y así sucesivamente.
```

Esto se llama generación **autorregresiva** (se alimenta de su propia salida). Palabra por palabra construye respuestas completas. **No hay una base de datos de respuestas: todo se genera token a token**.

4.2 La temperatura: creatividad vs. precisión

Temperatura	Comportamiento	Ideal para
Baja (0 – 0.3)	Elige casi siempre el token más probable. Precisa, determinista, repetible.	Datos, código, hechos
Alta (0.8 – 1.5)	Se arriesga con tokens menos probables. Creativa y variada.	Historias, lluvia de ideas

4.3 ¿Por qué a veces "alucina"?

Como **siempre** predice el siguiente token plausible, a veces genera algo que **suena bien pero es falso** (una "alucinación"). El modelo no tiene un verificador de verdad incorporado.

REGLA DEL APRENDIZ DE IA

Un LLM es un asistente brillante que puede equivocarse con total seguridad. **Siempre verifica los datos críticos** (fechas, cifras, nombres, citas legales).

4.4 El prompt: cómo le hablamos

El **prompt** es todo el texto de entrada que le damos. Un buen prompt produce una mejor respuesta. Cuatro claves de **ingeniería de prompts**:

1. **Da contexto y rol**: "Eres un profesor de matemáticas de primaria..."
2. **Sé específico**: di qué quieres, en qué formato y con qué longitud.
3. **Da ejemplos** (few-shot): muéstrale 1–2 ejemplos de lo que esperas.
4. **Pídele pasos**: "explica paso a paso" suele mejorar el razonamiento.

5. Chunking + RAG: darle nuestros documentos al LLM

Problema: el LLM no conoce tus documentos privados (el manual de tu empresa, tus apuntes) y no puedes meter un libro entero por el límite de tokens. **Solución: RAG.**

RAG = Retrieval-Augmented Generation (Generación Aumentada por Recuperación). La idea: *primero buscamos los fragmentos relevantes de nuestros documentos y se los pasamos al LLM junto con la pregunta.* El LLM responde **basándose en ellos**. Es como darle a un experto los apuntes correctos justo antes de que responda.

5.1 El pipeline RAG paso a paso

FASE DE PREPARACIÓN (una sola vez):

Documento grande

| 1. CHUNKING: partir en trozos (chunks)

▼

[chunk1][chunk2][chunk3]...

| 2. EMBEDDINGS: vectorizar cada chunk

▼

[vec1][vec2][vec3]... → guardar en una BASE DE DATOS VECTORIAL

FASE DE CONSULTA (cada pregunta):

Pregunta del usuario

| 3. vectorizar la pregunta

▼

[vec_pregunta]

| 4. BUSCAR los chunks más CERCANOS (similitud)

▼

chunks relevantes + pregunta → 5. PROMPT al LLM → respuesta fundamentada

5.2 ¿Qué es el chunking y por qué importa?

Chunking = partir un documento grande en trozos manejables (chunks) antes de vectorizarlos.

- Si el chunk es **muy grande**: mezcla varios temas y el embedding se vuelve "borroso".
- Si es **muy pequeño**: pierde contexto.
- Se suele usar **solapamiento (overlap)**: los chunks comparten un poco de texto en los bordes para no cortar ideas a la mitad.

"...la fotosíntesis convierte luz en energía. Las plantas usan clorofila..."

┌──────── chunk 1 ─────────┐

┌──────── chunk 2 ─────────┐ (con solapamiento)

5.3 Base de datos vectorial

Es una base de datos especializada en **guardar vectores** y **buscar los más parecidos rapidísimo**. Ejemplos: Chroma, FAISS, Pinecone, Weaviate. Es el "cerebro de memoria" de muchos asistentes de IA.

5.4 Práctica: mini-RAG casero

Ya tienes `clase-2/codigo/mini_rag.py`. Muestra la idea central de RAG sin base de datos: vectoriza unos chunks, vectoriza la pregunta y busca el más parecido.

```
chunks = [
    "El horario de atención del SENA es de lunes a viernes de 7am a 5pm.",
    "El curso de IA se dicta los sábados de 8am a 12m en Piedecuesta.",
    "Para certificarse se requiere el 80% de asistencia.",
]
emb_chunks = co.embed(texts=chunks, model="embed-multilingual-v3.0",
                      input_type="search_document").embeddings

pregunta = "¿A qué hora es la clase de inteligencia artificial?"
emb_preg = co.embed(texts=[pregunta], model="embed-multilingual-v3.0",
                    input_type="search_query").embeddings[0]

puntajes = [similitud(emb_preg, e) for e in emb_chunks]
mejor = int(np.argmax(puntajes))
print("Chunk más relevante:", chunks[mejor])
```

RESULTADO ESPERADO

Devuelve el chunk del **horario de la clase**. Esto es RAG en su forma más simple. Más adelante lo conectaremos a un LLM para que **responda con esa información**, formando un agente que "conoce" nuestros datos.

6. El pipeline completo de un LLM

TEXTO

- | Tokenización → [tokens] → [IDs numéricos]
- | Embeddings → cada token/frase = vector de significado
- | Transformer+Atención → entiende cada token según el contexto completo
- | Generación → predice el siguiente token, uno a uno (temperatura)



LENGUAJE NATURAL de salida

(+ RAG opcional: chunking → embeddings → BD vectorial → búsqueda → contexto extra)

Actividades de refuerzo

Realiza estas actividades para afianzar lo aprendido. Puedes escribir tus respuestas en los espacios en blanco.

1 Con mis palabras Fácil

Explica con tu propia analogía qué es un **token** y por qué un LLM "no ve palabras sino números". Menciona por qué existe un límite de contexto.

2 Cuenta tus tokens Fácil

Abre un tokenizador visual online, pega tres frases en español (una corta, una media y una larga) y anota cuántos tokens ocupa cada una. ¿Qué palabra se partió en más pedazos?

Frase	N.º de tokens

3 Midiendo el significado Media

Ejecuta `embeddings_similitud.py` con **tus propias palabras**. Elige 3 pares: uno **muy similar**, uno **medio** y uno **muy diferente**. Anota el puntaje de similitud coseno de cada par.

Par de palabras	Tipo esperado	Similitud (0 a 1)
	Muy similar	
	Medio	
	Muy diferente	

4 Detective del contexto Media

Escribe dos frases con una misma palabra ambigua (como "banco", "gato", "cura" o "vela") en las que signifique cosas distintas. Explica a qué otras palabras "presta atención" el modelo para adivinar el significado en cada caso.

5 Cazando alucinaciones Reto

Pídele a un LLM un dato muy específico y verificable (una fecha histórica, una cifra, una cita legal). Verifícalo en una fuente confiable. ¿Acertó o alucinó? Anota el prompt, la respuesta y tu verificación.

6 Diseña tu RAG Reto

Si quisieras un asistente que responda sobre el **reglamento del SENA**, ¿qué documentos "chunkearías"? Describe cómo los partirías y por qué usarías solapamiento. Prueba también a cambiar la `pregunta` en `mini_rag.py` y anota qué chunk devuelve.

Lista de verificación

- ☐ Sé qué es un token y por qué hay un límite de contexto.
- ☐ Puedo explicar qué es un embedding con la idea del "mapa del significado".
- ☐ Entiendo, a grandes rasgos, qué hace la atención en un Transformer.

- ☐ Sé explicar por qué un LLM alucina y por qué hay que verificar datos.
- ☐ Entiendo qué es chunking y para qué sirve RAG.
- ☐ Ejecuté `embeddings_similitud.py` y `mini_rag.py`.

SOLUCIÓN DE PROBLEMAS

ModuleNotFoundError: corre `pip install cohere numpy python-dotenv`. · **API key inválida:** revisa que `COHERE_API_KEY` esté en tu `.env`, completa y sin espacios. · **Se acabó el cupo:** espera unos minutos (la Trial key tiene límites) o genera una nueva. · **Puntajes raros:** asegúrate de usar `input_type="search_document"` para los textos.